PRD — Forja: runtime de execução da fábrica de software

Oct 31, 2026 · @Paulo

Resumo

A Forja é o runtime de execução da fábrica de software do FuseOne: um servidor MCP que recebe "implemente a issue X no repositório Y" e devolve uma PR com evidência verificável. A orquestração, a governança e o custo continuam no FuseOne; a Forja só faz o que o FuseOne hoje não faz — checkout, build, testes e serviços de apoio num ambiente efêmero e isolado.

Decisão pedida: aprovar a fase 0 como servidor MCP externo, que o FuseOne registra como qualquer outro, num repositório piloto de outra área nomeado antes da aprovação. A fase 0 inclui escolher e medir o agente que roda no workspace. Tornar a Forja o Level 2 oficial da FuseOne é decisão posterior e não bloqueia a fase 0.

Repositório piloto: **a definir — condição para aprovar**.

Problema e evidência

A fábrica do FuseOne v1 orquestra papéis sobre o board, mas nenhum papel executa código. A única ferramenta é o MCP do GitHub; o Developer só commita pela API ("Level 1") e o QA lê o CI em vez de rodar testes. A NT-013 já nomeia a lacuna: um "coding runtime atrás de uma ferramenta MCP" ("Level 2"), com build e testes antes da PR, ainda pendente.

O piloto do zen mostrou que a qualidade da fábrica mora justamente nessa execução:

- O desenvolvedor roda os gates do repositório (build, lint, testes unitários e de integração com Postgres real) e só abre a PR com exit code zero.
- O QA inverte a implementação, inteira e pela metade, e exige que os testes falhem; teste que não morde derruba a entrega.
- No primeiro lote, o revisor de segurança achou um HIGH e um MEDIUM que o QA não viu (PRs #1253 e #1254). Revisão de leitura e execução são complementares, não substitutas.

- Hoje cada pessoa pluga a própria máquina como worker, com a própria conta. Isso não escala para outras áreas nem dá isolamento real.

Sem um runtime compartilhado, cada área que quiser uma fábrica vai reconstruir worktree, gates e reserva de card por conta própria.

Objetivos e não-objetivos

Objetivos:

1. Qualquer repositório que declare um contrato `.forja.yaml` pode receber implementação e QA por execução, sem código específico da área na Forja.
2. Toda PR aberta pela Forja carrega evidência reproduzível: comandos rodados, exit codes, logs dos gates e o resultado da inversão.
3. Cada run roda num ambiente efêmero, sem credencial e com egress só para os proxies do plano de controle, e some ao terminar.
4. O FuseOne continua dono de agendamento, aprovação, custo e trilha; a Forja é só mais um servidor MCP classificado pelo Gate.
5. Nenhuma máquina pessoal no caminho: a identidade que commita é um GitHub App da instalação.

Não-objetivos:

- Orquestrar papéis, mover cards ou decidir prioridade. Isso é do FuseOne.
- Fazer merge. O merge segue a política de cada área no FuseOne.
- Escrever os prompts dos papéis de cada área. Eles vivem no repositório da área.
- Garantir correção além do que os gates do repositório seguram.
- Ler o mundo de produção (cluster, cofre, banco). Medição fica com ferramentas próprias de cada área.

Usuários e casos de uso

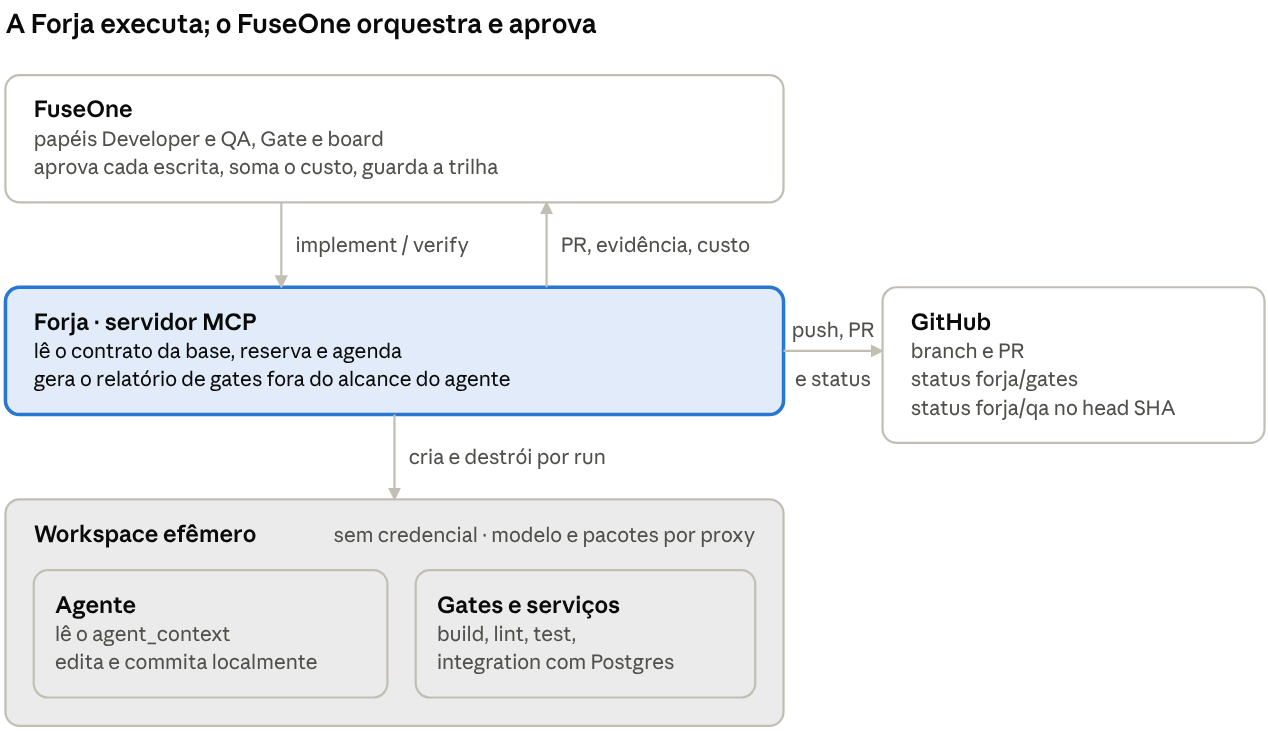
Usuário	O que quer	Como usa a Forja
Papel Developer do FuseOne	Implementar um card preparado	Chama `forja.implement` com issue, repositório e branch base; recebe a PR e a evidência

Usuário	O que quer	Como usa a Forja
Papel QA do FuseOne	Tentar derrubar a entrega	Chama `forja.verify` na PR; recebe veredito, inversões rodadas e quais testes morderam
Papel Developer, na rodada de correção	Responder a achado de review	Chama `forja.implement` na mesma branch com o achado como entrada
Mantenedor do repositório da área	Ligar a fábrica no seu repo	Escreve o `.forja.yaml`, roda `forja.doctor` e ajusta até os gates passarem na `main`
Dono da plataforma FuseOne	Operar e cobrar	Vê runs, duração, custo de computação e falhas por repositório

O caso de uso que decide o produto é o do mantenedor: se ligar um repositório novo levar mais que um dia, a adoção por outras áreas não acontece.

Solução

A Forja tem duas partes: um plano de controle (o servidor MCP, que reserva, agenda e julga) e workspaces efêmeros (onde o agente trabalha). O julgamento fica no plano de controle, fora do alcance do agente.



arquitetura · FuseOne, Forja, workspace e GitHub

O FuseOne chama e aprova; a Forja cria o workspace, roda o agente e os gates, e publica no GitHub um status que o agente não consegue escrever.

Fluxo de `forja.implement`:

1. Reserva a issue em `refs/forja/claims/<n>`; se a ref existe, recusa.
2. Lê o `.forja.yaml` do commit da branch base, nunca da árvore do agente, e fixa esse SHA na evidência.
3. Clona o repositório no plano de controle e monta a árvore no workspace, com a imagem e os serviços do contrato. O workspace não recebe credencial.
4. Roda o agente com o enunciado da issue e o `agent_context`. Ele fala com o modelo e com os registries só pelos proxies do plano de controle e commita localmente.
5. O plano de controle roda os gates do contrato da base sobre a árvore do agente e guarda log cru e exit code de cada um.
6. Gate vermelho volta ao agente até o limite de tentativas; esgotado, o run termina como falha com a evidência.
7. Tudo verde: o plano de controle faz push, abre a PR com o relatório e publica `forja/gates` no head SHA.
8. Destrói o workspace, libera a reserva e devolve PR, evidência e custo ao FuseOne.

`forja.verify` também roda um agente, o de QA, com custo próprio. Ele escolhe as inversões como no piloto: a metade da mudança que uma regressão real desfaria e o complemento do que o desenvolvedor declarou testar. A inversão toca só código de produção; os testes novos ficam, e a Forja roda os gates do contrato da base e exige vermelho. Reverter o diff inteiro não serve: apaga os testes novos junto e nada fica vermelho.

O veredito tem três valores:

- **Morde:** toda inversão deixou algum gate vermelho. Publica `forja/qa` verde.
- **Não morde:** alguma inversão passou verde. Publica `forja/qa` vermelho com a inversão que escapou.
- **Não verificável:** o diff só toca caminhos excluídos ou não tem código de produção. Publica `forja/qa` neutro, nunca verde, e o merge fica com uma pessoa.

Contrato do repositório

O `.forja.yaml` na raiz é a única coisa que a área escreve para ligar a Forja. Ele declara como o repositório se constrói e se prova; a Forja não adivinha nada.

```
version: 1
image: golang:1.24            # imagem base do workspace
setup:
  - make deps
services:                      # sobem como sidecars efêmeros
  postgres: { image: postgres:17, env: { POSTGRES_PASSWORD: test } }
registries:                    # leitura de dependência privada, pelo proxy
  - ghcr.io/minha-org
  - npm.pkg.github.com
gates:                         # ordem importa; todos precisam sair 0
  - name: build
    run: make build
  - name: lint
    run: make lint
  - name: test
    run: make test
  - name: integration
    run: make integration-test
    needs: [postgres]
verify:
  exclude: ["**/*.md", "migrations/**"]  # diff só aqui = não verificável
```

```
proof_paths:                    # aparato de prova: mudança fica visível e
pede merge humano
  - .forja.yaml
  - Makefile
  - scripts/**
  - "**/testdata/**"
  - "**/*_test.go"              # só alteração ou remoção; teste novo não
conta
human_merge_paths:              # sinal para a política de merge do FuseOne
  - .github/**
limits:
  wall_clock: 30m
  max_diff_lines: 400
agent_context:                  # o que o agente lê antes de codar
  - CLAUDE.md
  - CONTRIBUTING.md
```

Regras do contrato:

- O plano de controle lê o `.forja.yaml` do commit da branch base. Uma mudança do agente nesse arquivo não vale para o run que a fez.
- Gates são comandos do próprio repositório. A Forja só lê exit code e guarda o log cru; nunca interpreta saída de teste.
- O contrato da base não impede o agente de mudar o que os gates executam: Makefile, scripts, helpers e fixtures de teste. A Forja não promete que o agente não afrouxa a prova; promete que isso fica visível. Toda mudança em `proof_paths`, e toda alteração ou remoção de teste existente, abre a seção "aparato de prova" da evidência e manda a PR para merge humano.
- `registries` lista onde o `setup` busca dependência privada. A credencial de leitura fica no proxy; o workspace só enxerga o proxy.
- `forja.doctor` roda o contrato contra a `main` e recusa ligar o repositório enquanto algum gate falhar lá, inclusive o `setup` com os `registries` declarados.

Segurança e isolamento

O agente dentro do workspace é tratado como código não confiável: ele lê issue e PR escritas por pessoas e executa o que o repositório manda. O isolamento é do runtime, nunca de regra no prompt.

- **Um workspace por run.** Container ou microVM nova, destruída ao fim, sem volume compartilhado entre runs ou repositórios.
- **Nenhuma credencial no workspace.** Checkout, push e PR são feitos pelo plano de controle com o token do GitHub App, escopado ao repositório e à duração do run. Serviços de apoio usam credenciais descartáveis geradas no boot.
- **Modelo por proxy.** O agente fala com a API do modelo por um proxy no plano de controle, que guarda a chave, aplica o teto de tokens e mede o custo do run.
- **Dependência privada por proxy.** Os `registries` do contrato passam por um proxy com credencial só de leitura, por área. Isso alarga a superfície: o agente lê pacotes privados da organização, e o que segura a exfiltração é o egress.
- **Egress só para os proxies.** O workspace alcança o proxy do modelo e o de pacotes; o resto é negado e logado.
- **Push só em branch própria.** O App não tem permissão de bypass; a proteção da `main` continua valendo.
- **Evidência assinada.** O relatório de gates é gerado pelo plano de controle, fora do alcance do agente, e publicado como commit status `forja/gates` no head SHA. O agente não consegue forjar um gate verde.
- **Reserva atômica.** Um run por issue, por ref no repositório (`refs/forja/claims/<n>`), com lease que vence e devolve o card — o mesmo mecanismo medido no piloto.

O revisor de segurança por leitura continua sendo papel do FuseOne; a Forja não o substitui.

Integração com o FuseOne

A Forja entra como um servidor MCP registrado por área, ao lado do MCP do GitHub. Nada muda no modelo de papéis, board ou Gate.

Tool	Classe no Gate	Quem chama	Efeito
`forja.doctor`	read	mantenedor, Triage	roda o contrato na `main`, sem push
`forja.plan`	read	Developer	checkout + leitura do `agent_context`; devolve o que o agente vê, sem escrever

Tool	Classe no Gate	Quem chama	Efeito
`forja.implement`	write	Developer	branch, commits, gates e PR com `forja/gates`
`forja.verify`	write	QA	agente de QA, inversões e gates; publica `forja/qa` no head SHA
`forja.status`	read	qualquer papel	estado, log e custo de um run

- **Identidade:** um GitHub App da Forja por instalação, instalado só nos repositórios ligados. O App do papel no FuseOne continua assinando comentários e reviews.
- **Aprovação:** `implement` e `verify` são write; em copilot, esperam uma pessoa como qualquer escrita. Uma área que queira autonomia decide isso no FuseOne, não na Forja.
- **Custo:** a Forja reporta minutos de computação e tokens do agente por run; o FuseOne soma na área e aplica o teto por run que já existe.
- **Merge:** `forja/gates` e `forja/qa` são os sinais que a ruleset da `main` e o futuro `merge_if_ready` da NT-013 podem exigir, cada um fixado ao seu produtor.

Métricas de sucesso

As metas abaixo são hipóteses para calibrar no piloto; nenhuma foi medida ainda.

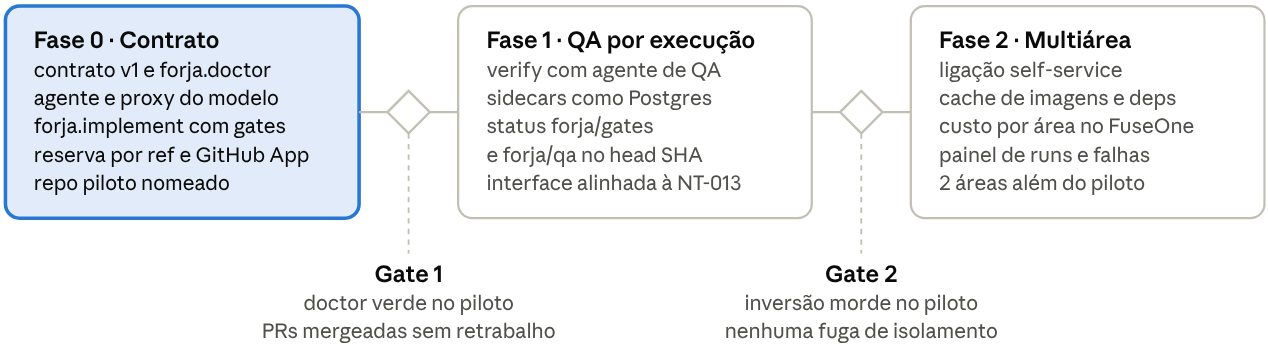
Métrica	Como mede	Meta inicial
Tempo para ligar um repositório	do primeiro `.forja.yaml` ao `forja.doctor` verde	⩽ 1 dia
PRs da Forja mergeadas sem retrabalho humano no código	PRs mergeadas cujo diff final é só do agente	a medir no piloto
Inversões que mordem	% de PRs em que o QA viu teste vermelho ao inverter	a medir no piloto
PR verde e errada	PRs revertidas ou corrigidas em até 14 dias após merge	tendência de queda
Áreas ativas	áreas com pelo menos um run por semana	2 áreas além do piloto em 6 meses

Métrica	Como mede	Meta inicial
Fuga de isolamento	eventos de egress negado com origem no agente, investigados	0 sem explicação

Fases de entrega

A fase 0 é a decisão pedida: contrato, runtime mínimo e a escolha medida do agente, num repositório de outra área nomeado,, antes de qualquer investimento em multiárea.

Três fases, cada uma liberada por um gate medido no piloto



roadmap · 3 fases, 2 gates

As fases não têm data: a duração de cada uma sai do que a fase 0 medir. Nenhuma fase começa sem o gate anterior verde.

Riscos e questões em aberto

O maior risco é de produto, não de engenharia: a Forja entrega só o que os gates do repositório seguram. Num repositório com pouca cobertura, ela produz PR verde e errada com cara de confiável.

Risco	Mitigação
Repositório com gates fracos gera PR verde e errada	`forja.verify` com agente de QA e veredito "não verificável" quando não há o que inverter; `doctor` reporta cobertura; piloto escolhido com testes medianos de propósito

Risco	Mitigação
Agente afrouxa teste ou gate para passar	contrato lido da base; mudança em `proof_paths` ou em teste existente aparece na evidência e manda a PR para merge humano
Dependência privada pelo proxy amplia o que o agente lê	credencial só de leitura e por área; egress só para os proxies; acesso logado por run
Verify com agente encarece cada PR	custo do QA medido à parte na fase 1; teto por run no FuseOne
Serviços de apoio pesados (Kafka, emuladores) estouram o tempo do run	`limits.wall_clock` por repositório; imagens pré-aquecidas; fase 2 trata cache
Custo de computação sem dono	reporte por run ao FuseOne, que já tem orçamento por área
Sobreposição com o Level 2 que a FuseOne venha a fazer	interface MCP alinhada à NT-013 na fase 1; adoção como Level 2 oficial decidida depois

Questões em aberto:

- [ ] Qual repositório de outra área entra no piloto, e quem é o mantenedor? Bloqueia a aprovação.
- [ ] Workspace em container com gVisor ou em microVM (Firecracker)? Decide o custo de serviços como Postgres dentro do run.
- [ ] Onde a Forja roda: no mesmo cluster do FuseOne ou em pool dedicado? Isolamento de rede pede pool dedicado.
- [ ] A FuseOne adota a Forja como Level 2 oficial? Não bloqueia a fase 0.

Qual agente roda no workspace (Claude Code headless ou um loop próprio sobre a API) não é questão em aberto: é o trabalho da fase 0, porque decide custo, controle de ferramentas e a forma do proxy do modelo.